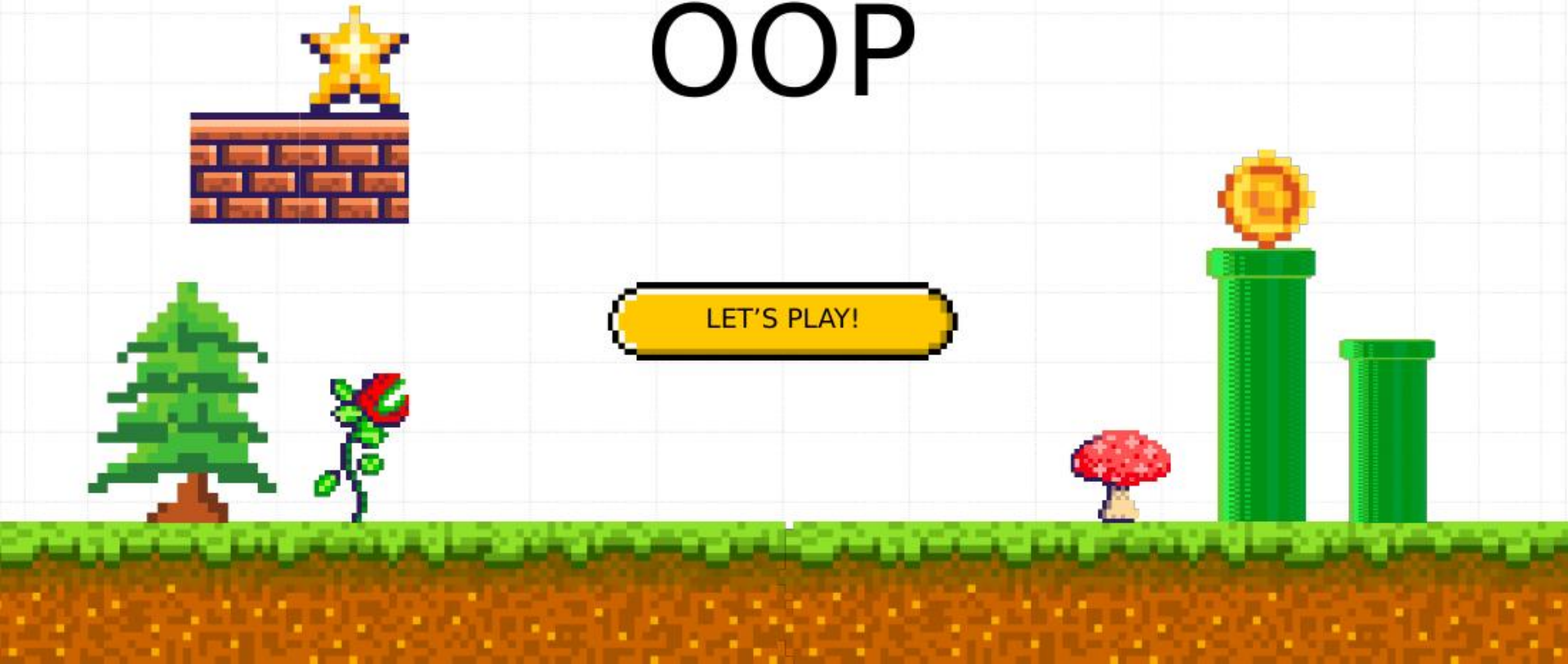
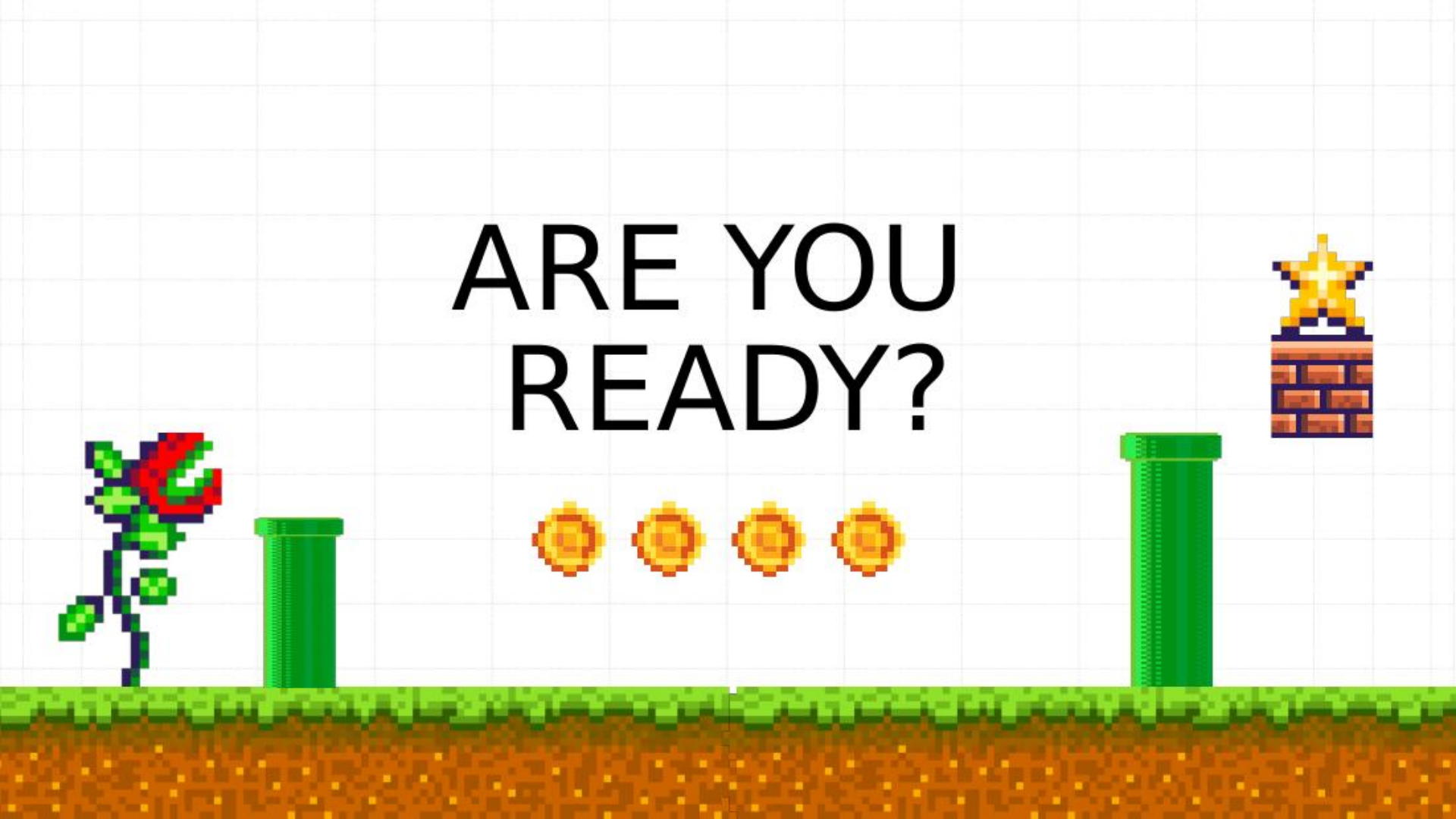
OOP



ARE YOU READY?

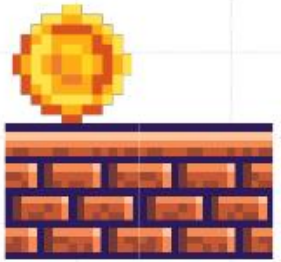


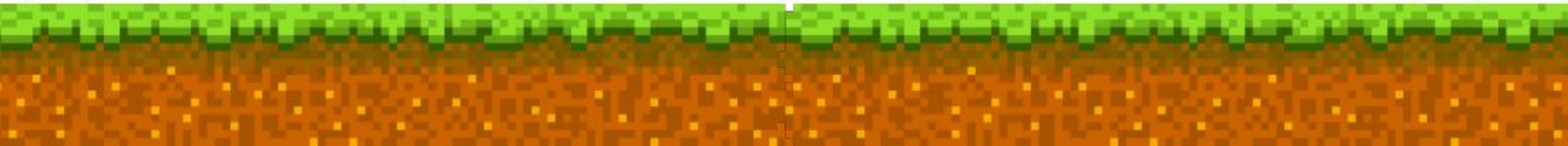
OBJECTIVES

- Understanding the core concept of OOP
- Understand object instantiation and member access
- Apply principles of inheritance, polymorphism, encapsulation and abstraction

Answer the question

What is OOP?





OOP

- OOP stands for Object Oriented Programming
- Object-oriented programming is about creating objects that contain both data and methods.

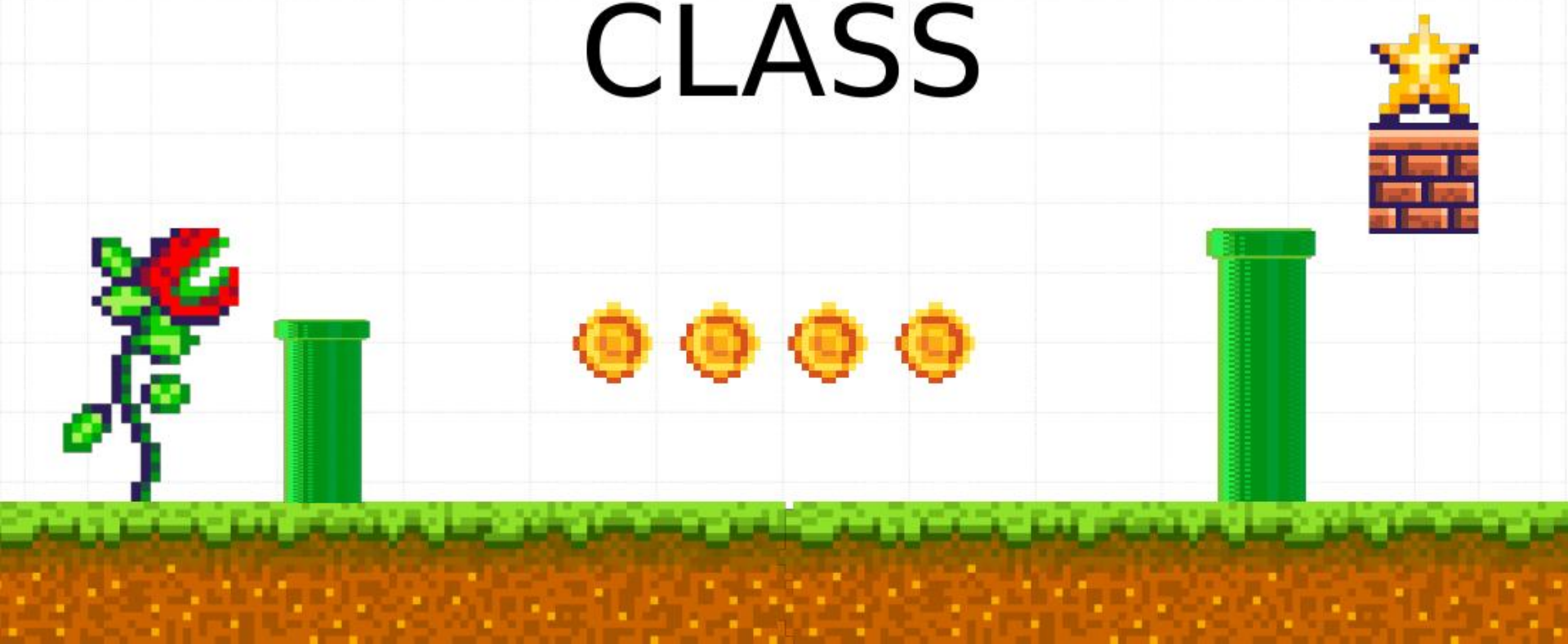
Complete the word

Answer:

W H Y
O O P ?

- It is faster and easier to execute.
- Provides a clear structure of the programs
- Helps to keep the Java code easier to maintain, modify and debug
- It makes it possible to create full reusable applications with less code and shorter development time.

CLASS



- A class can be defined as a template/blueprint that describes the behavior/state that the object of its type support.
- It helps us to bind data and methods together, making the code reusable.

VARIABLE TYPES INSIDE A CLASS

- **Local variables** − Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.
- **Instance variables** − Instance variables are variables within a class but outside any method. These variables are initialized when the class is instantiated. Instance variables can be accessed from inside any method, constructor or blocks of that particular class.
- **Class variables** − Class variables are variables declared within a class, outside any method, with the static keyword.

Complete the word

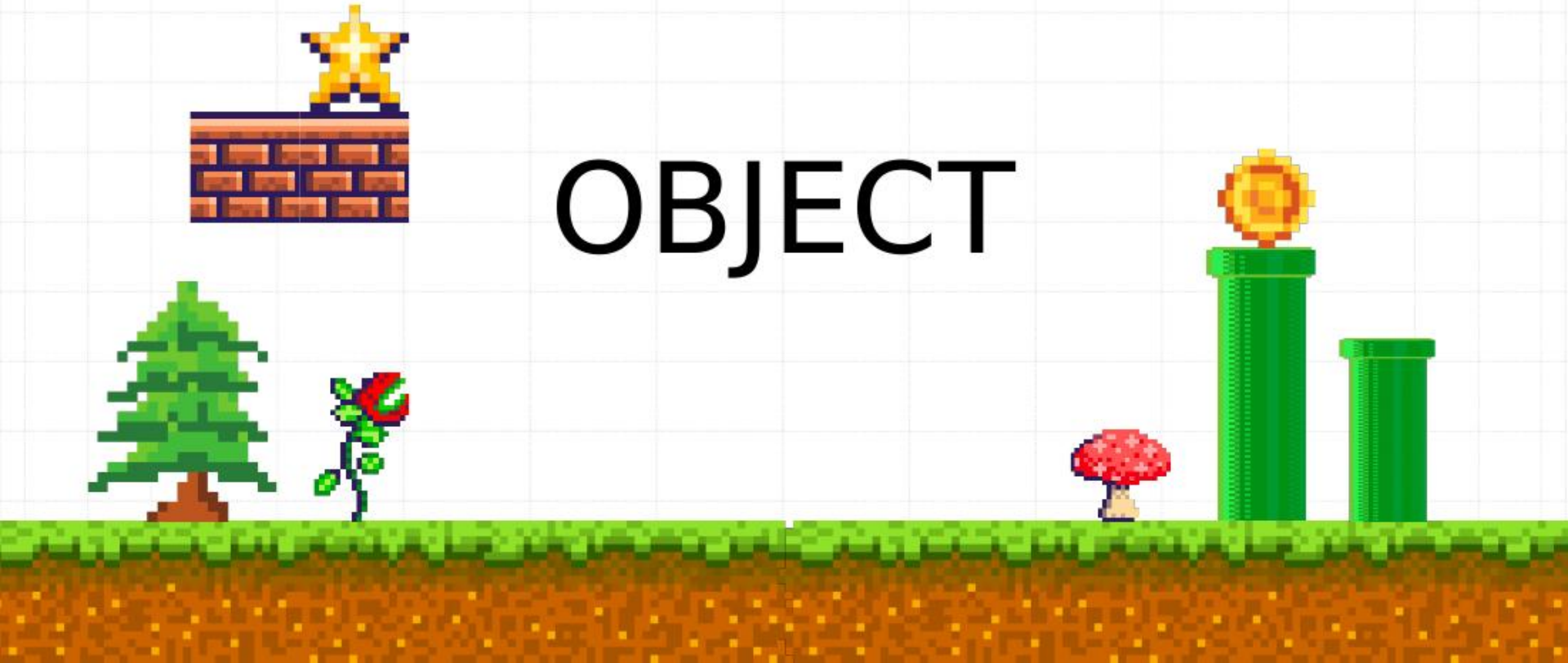
Answer:

```
Modifiers className {

//Attributes
//Method

}
```

```java
public class Dog {

    // attribute
    String name;

    // method
    void walk(){
        System.out.print("");
    }
}
```

```java
public class house {

    // State of the class house
    String color;
    String windowType;

   // behavior or the method of the class house
    void shelter(){
        System.out.print("Welcome home");
    }
}
```

OBJECT

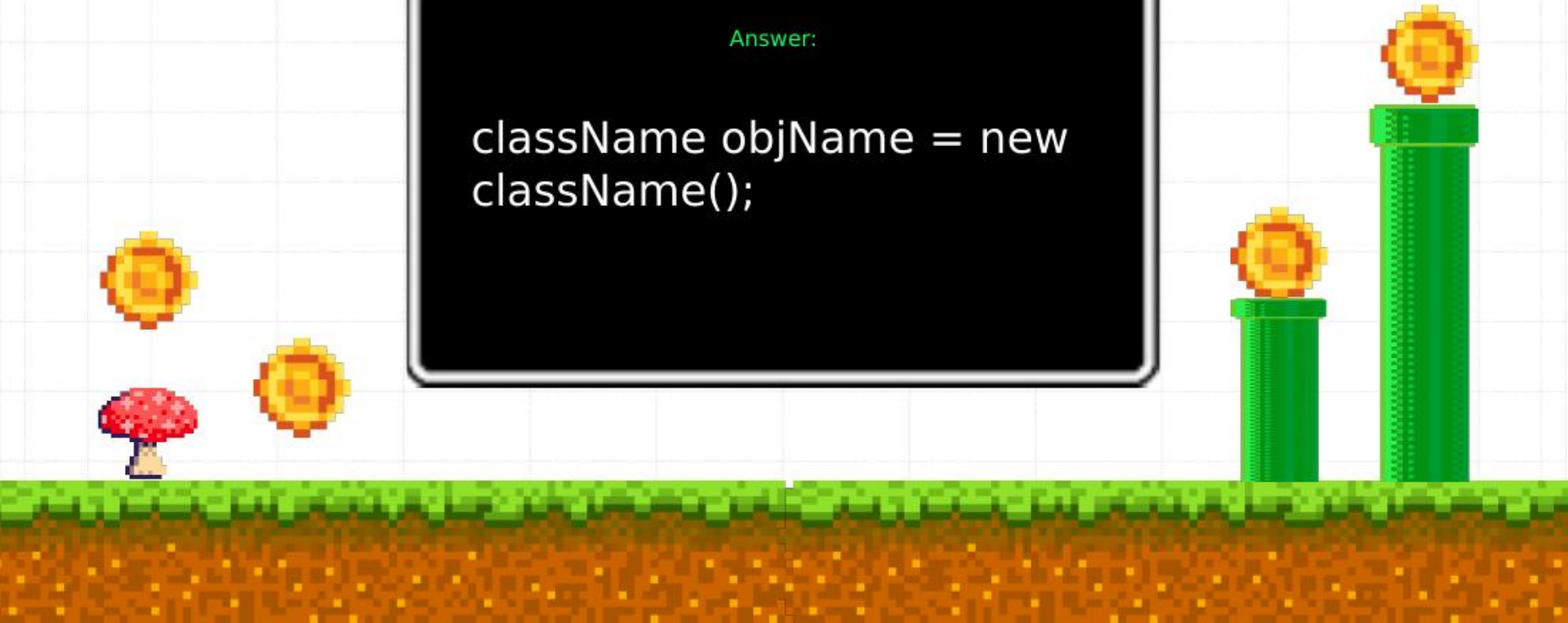


- An Object is an instance of a class, which contains data and methods working on that data. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.
- Objects have states and behaviors. Example: A dog has states - color, name, breed as well as behaviors – wagging the tail, barking, eating. An object is an instance of a class.
- Object consist of three things:
- Name: This is a variable name that represents the object
- Member data: The data that describes the object
- Member methods: Behavior that describes the object

Complete the word

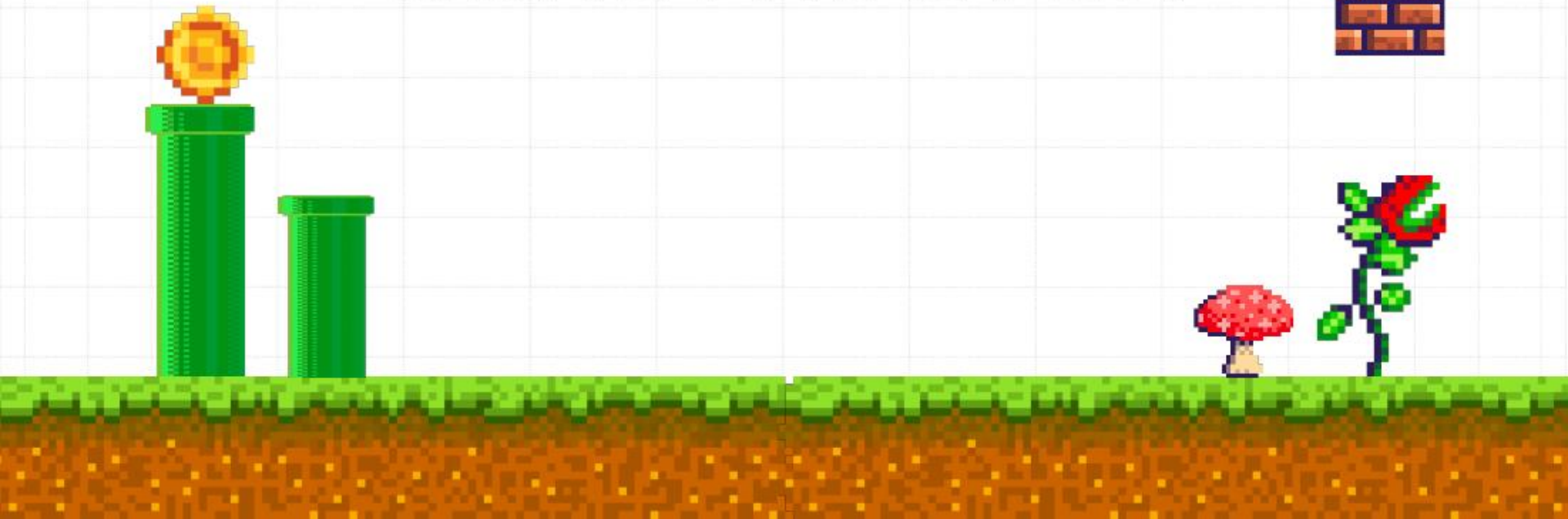
Answer:

```
className objName = new
className();
```



```
    public static void main(String[] args) {

        // creating an object
        Cat siamese = new Cat();    // siamese is an object
        siamese.eat();

    }

}
```

CONSTRUCTOR



Introduction to Classes

Constructors

A *constructor* initializes an object immediately upon creation. It has the same name as the class in which it resides and is syntactically similar to a method.

Once defined, the constructor is automatically called when the object is created, before the **new** operator completes.

Constructors look a little strange because they have no return type, not even **void**.

Introduction to Classes

Constructors

It is the constructor’s job to initialize the internal state of an object so that the code creating an instance will have a fully initialized, usable object immediately.

RULES IN CREATING A CONSTRUCT

Constructors

1. The name of the constructor should be the same as the class
2. **A java constructor must not have a return type.**

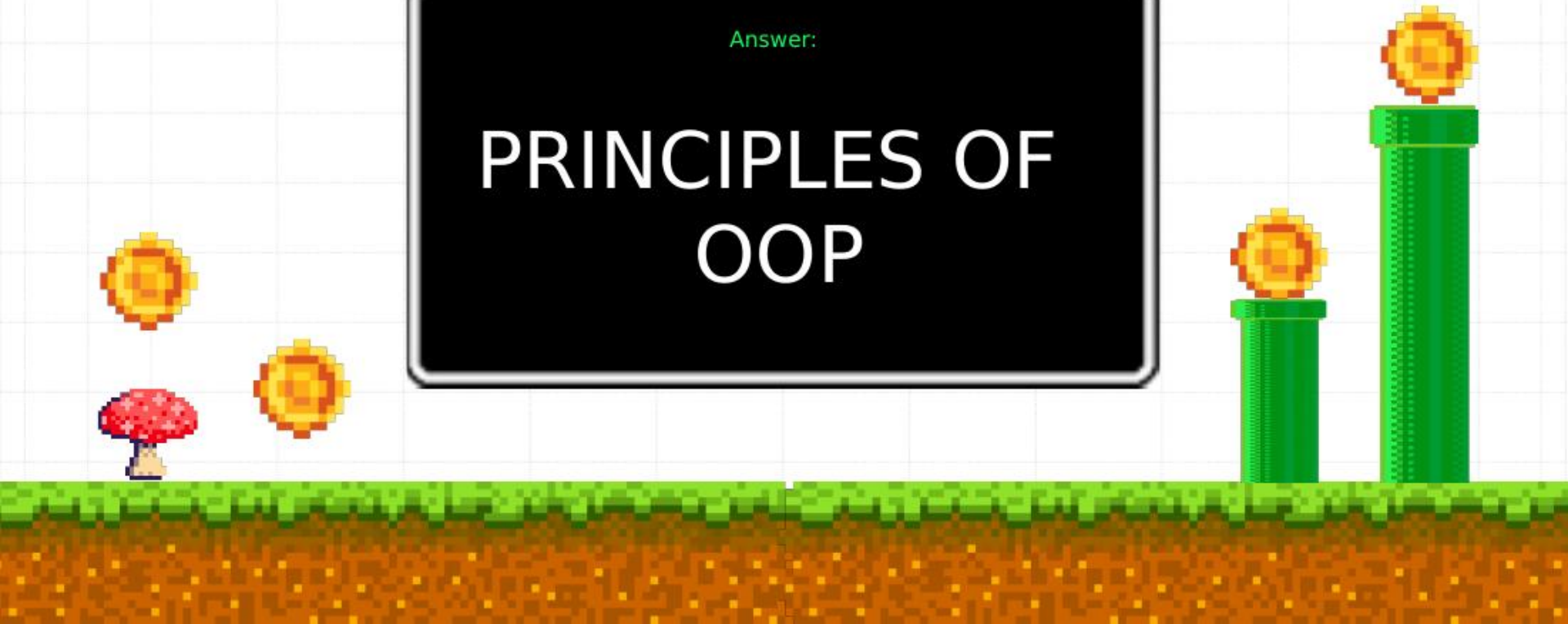
EXAMPLE CODE

```
public class house {

    String color;
    String windowType;

    // ang constructor dapat the same name sa class and no return value
    // we can declare constructor without parameter
    house(){
        System.out.print("New Color");
    }

    // constructor with parameter
    house(String color){
     this.color=color;
     System.out.println("The color of this house is "+color);
   }
```

Complete the word

Answer:

PRINCIPLES OF OOP



Understanding the Basics

OOP Principles

- **Encapsulation**

Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.

Understanding the Basics

OOP Principles

- **Encapsulation**

One way to think about encapsulation is as a protective wrapper that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper.

Access to the code and data inside the wrapper is tightly controlled through a well-defined interface.

Understanding the Basics

OOP Principles

➢ Encapsulation

In Java, the basis of encapsulation is the **class.**

A ***class*** defines the structure and behavior (data and code) that will be shared by a set of objects. Each object of a given class contains the structure and behavior defined by the class, as if it were stamped out by a mold in the shape of the class. For this reason, **objects** are sometimes referred

Example Code

```
public class person {

    // using of private modifier for data hiding
    private int age;

    // getter method - read only
    public int getAge(){
        return age;
    }

    //setter method - write only
    public void setAge(int age){
            this.age=age;

    }
}
```

Example Code

```
public class OOP {

    public static void main(String[] args) {

     person p1 = new person(); // create object of the class person

     p1.setAge(25); // using setter method to write age

     System.out.println("Age: "+p1.getAge()); // using getter method to access the set age

    }
}
```

Understanding the Basics

OOP Principles

- **Inheritance**

Inheritance is the process by which one object acquires the properties of another object.

This is important because it supports the concept of hierarchical classification.

A deeply inherited subclass inherits all of the attributes from each of its ancestors in the *class hierarchy*.

EXAMPLE CODE

```
public class Animal {      // superclass or mother class

    String name;

    public void walk(){
        System.out.println("I can walk");
    }
}
```

EXAMPLE CODE

```java
public class Dog extends Animal {  // subclass or child class - inherits all properties of mother class

    @Override  // to specify that the walk method was also present in the superclass
    public void walk(){
        super.walk();  // the use of super keyword para ma read ug una ang walk method sa mother class
         System.out.println("I am walking");
    }

    public void display(){
        System.out.println("My name is "+name);
    }

}
```

EXAMPLE CODE

```
public class OOP {


    public static void main(String[] args) {


      Dog labrador = new Dog();  //create object for the class dog

      labrador.name ="Ash"; // inherited the name property from the mother class
      labrador.walk();
      System.out.println("My name is: "+ labrador.name);

    }
}
```

Understanding the Basics

OOP Principles

- **Polymorphism**

Polymorphism (from Greek, meaning “many forms”) is a feature that allows one interface to be used for a general class of actions.

The specific action is determined by the exact nature of the situation.

Understanding the Basics

OOP Principles

- **Polymorphism**

The concept of polymorphism is often expressed by the phrase **“one interface, multiple methods.”** This means that it is possible to design a generic interface to a group of related activities.

This helps reduce complexity by allowing the same interface to be used to specify a *general class of action*.

EXAMPLE CODE

Start Page | OOP.java | Shape.java | rectangle.java | square.java | hexagon.java

Source | History

```
15      public static void main(String[] args) {
16
17              // create object for each sub class
18              rectangle r1 = new rectangle();
19
20              square s1 = new square();
21
22              hexagon h1 = new hexagon();
23
24              // objects respond differently to its mother class
25              r1.sides();
26              s1.sides();
27              h1.sides();
28
```

Output - OOP (run)

```
run:
Rectangle - 4 sides
Square - 4 equal sides
Hexagon - 6 equal sides
BUILD SUCCESSFUL (total time: 0 seconds)
```

```java
public class Shape { // parent class

    public void sides(){
        System.out.println("Different shapes");
    }

    // method overloading
    public void display(){      // without parameter
        for(int i=0; i<10;i++){
            System.out.print("*");
        }
    }
      public void display(char symbol){  // with parameter
        for(int i=0; i<10;i++){
            System.out.print(symbol);
        }

    }

}
```

```java
public class rectangle extends Shape { //subclass

    @Override
     public void sides(){
        System.out.println("Rectangle - 4 sides");
    }

}
```

```java
public class square extends Shape { //subclass

    @Override
     public void sides(){
        System.out.println("Square - 4 equal sides");
    }
}
```

```java
public class hexagon extends Shape {  //subclass

    @Override
     public void sides(){
        System.out.println("Hexagon - 6 equal sides");
    }
}
```

Understanding the Basics

Abstraction

An essential element of object-oriented programming is *abstraction*. Humans manage complexity through abstraction.

For example, people do not think of a car as a set of tens of thousands of individual parts. They think of it as a well-defined object with its own unique behavior.

Understanding the Basics

Abstraction

A powerful way to manage abstraction is through the use of hierarchical classifications.

This allows you to layer the semantics of complex systems, breaking them into more manageable pieces.

Understanding the Basics

Abstraction

From the outside, the car is a single object. Once inside, you see that the car consists of several subsystems: steering, brakes, sound system, seat belts, heating, cellular phone, and so on.

In turn, each of these subsystems is made up of more specialized units.

Understanding the Basics

Abstraction

Hierarchical abstractions of complex systems can also be applied to computer programs.

The data from a traditional process-oriented program can be transformed by abstraction into its component objects.

A sequence of process steps can become a collection of messages between these objects.

Understanding the Basics

Abstraction

Thus, each of these objects describes its own unique behavior.

You can treat these objects as concrete entities that respond to messages telling them to *do something*.

This is the essence of object-oriented programming.

```java
abstract class Animal {     // abstract class - parent class

    abstract void eat();  // abstract method - with this method declaration all child class should implement this eat() method
                          // pag dili ma implement ang abstract method mag error ang Cat and Dog class

    void movement(){        // regular method
        System.out.println("I can do whatever movement");
    }

}
```

```java
public class Cat extends Animal {

    @Override
    void eat(){    // implementing the  abstract method
        System.out.println("I can eat cat food");
    }

}
```

```java
public class Dog extends Animal{
    @Override
    void eat(){ // implementing the  abstract method
        System.out.println("I can eat dog food");
    }
}
```

Main Method

```
public class Abstract_OOP {

    public static void main(String[] args) {

        Cat siamese = new Cat();
        siamese.eat();

        Dog labrador = new Dog();
        labrador.eat();

    }

}
```

THANKS FOR PLAYING

END